

PWA

CI/CD

INFORME DE LABORATORIO N.º 6

Desarrollo de una Progressive Web App e implementación de su despliegue automatizado
mediante contenedores y CI/CD

Asignatura	Programación Integrativa de Componentes Web
Estudiante	Shadya Reyes
Docente	Ing. Kevin Chuquitarco, Mgtr.
NRC / Nivel	29544 / Sexto nivel
Práctica	Laboratorio N.º 6
Fecha	25 de julio de 2026

Repositorio configurado: github.com/ShadyNico/P2_Lab1_ReyesShadya_ComponentesReutilizables

1. Resumen

Se integró una aplicación React con una API REST construida con Node.js, Express, PostgreSQL y JWT. El frontend permite registrar usuarios, iniciar sesión y administrar productos mediante operaciones de consulta, registro, edición y eliminación. La ruta de productos está protegida y Axios incorpora automáticamente el token Bearer en cada solicitud.

La aplicación se convirtió en PWA mediante vite-plugin-pwa, un Web App Manifest explícito, iconos de 192 y 512 píxeles y un Service Worker generado con Workbox. Finalmente, se añadieron Dockerfiles, Docker Compose y un flujo de GitHub Actions para validar el código, construir las imágenes y publicarlas en Docker Hub cuando existan los secretos requeridos.

2. Objetivos

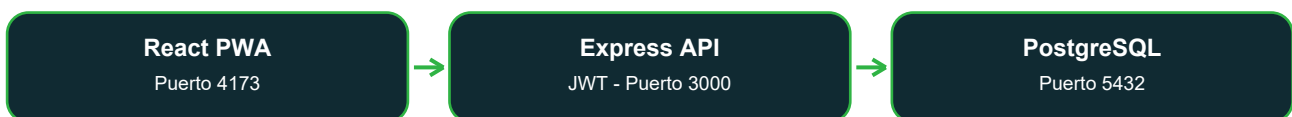
2.1 Objetivo general

Implementar una PWA full stack y automatizar su construcción y publicación mediante Docker y GitHub Actions, aplicando autenticación JWT, persistencia en PostgreSQL y buenas prácticas de integración continua.

2.2 Objetivos específicos

- Centralizar el consumo de la API con Axios y cabeceras Bearer.
- Implementar registro, login y almacenamiento controlado del JWT.
- Proteger el módulo de productos y completar su CRUD.
- Generar el manifiesto, Service Worker e iconos instalables.
- Contenerizar frontend, backend y PostgreSQL.
- Construir un pipeline CI/CD con publicación de imágenes en Docker Hub.

3. Arquitectura implementada



Docker Compose crea una red privada para los tres servicios. El navegador consume el frontend y la API a través de los puertos publicados; el backend accede a PostgreSQL mediante el nombre de servicio database. La base de datos se inicializa con backend/database/script.sql y conserva sus datos en el volumen postgres_data.

4. Desarrollo de autenticación y productos

4.1 Servicio HTTP centralizado

El archivo `src/services/api.js` define la URL de la API, un tiempo de espera de 10 segundos y dos interceptores. El interceptor de solicitud agrega el JWT a la cabecera `Authorization` y el interceptor de respuesta elimina una sesión expirada cuando la API devuelve 401.

```
Authorization: Bearer <token>
VITE_API_URL=http://localhost:3000
```

4.2 Registro e inicio de sesión

El registro valida nombre, correo, contraseña y confirmación. La API evita correos duplicados, cifra la contraseña con `bcrypt` y asigna el rol `CLIENTE`. El login compara el hash, genera un JWT de una hora y devuelve los datos públicos del usuario.

Elemento	Implementación	Resultado
Registro	POST /auth/register	Usuario almacenado con hash <code>bcrypt</code>
Login	POST /auth/login	JWT y usuario devueltos
Persistencia	<code>localStorage</code>	token y usuario disponibles
Protección	<code>ProtectedRoute</code>	Redirección a /login sin JWT

4.3 CRUD de productos

La página `src/pages/productos/Productos.jsx` presenta un formulario reutilizable para alta y edición, una cuadrícula de productos, confirmación antes de eliminar y mensajes visibles para cada resultado. Todas las rutas del router de productos ejecutan el middleware de autenticación.

Operación	Endpoint	Comportamiento
Consultar	GET /productos	Lista ordenada por ID descendente
Registrar	POST /productos	Valida nombre, descripción, precio y stock
Editar	PUT /productos/:id	Carga el registro y actualiza la lista
Eliminar	DELETE /productos/:id	Solicita confirmación y actualiza la lista

5. Conversión a Progressive Web App

Vite integra vite-plugin-pwa con estrategia de actualización automática. El manifiesto define nombre, nombre corto, idioma, URL inicial, alcance, modo standalone, colores e iconos maskable. El Service Worker precachea el shell de la aplicación y usa NetworkFirst para las consultas GET de productos.

Recurso	Ubicación / valor	Verificación
Manifest	public/manifest.webmanifest	HTTP 200 y enlace en index.html
Service Worker	dist/sw.js	Generado por Workbox, HTTP 200
Icono pequeño	192 x 192 PNG	Dimensiones verificadas
Icono grande	512 x 512 PNG	Dimensiones verificadas
Modo de pantalla	standalone	Listo para instalación
Tema	#071821	Coincide con meta theme-color

Comandos de verificación

```
npm run build
npm run preview
```

La compilación generó nueve entradas de precaché, sw.js, el archivo de Workbox, el manifiesto y todos los iconos. La interfaz se revisó en el navegador con vista de escritorio y un viewport móvil de 390 x 844; no se detectó desplazamiento horizontal ni errores de consola.

6. Contenerización

Se crearon Dockerfiles independientes basados en Node 22 Alpine. El frontend compila la PWA y la expone con Vite Preview en el puerto 4173. El backend instala únicamente dependencias de producción, ejecuta con un usuario sin privilegios y publica el puerto 3000.

```
docker compose up -d --build
```

Servicio	Imagen	Puerto	Estado final
frontend	Node 22 Alpine	4173	Healthy
backend	Node 22 Alpine	3000	Healthy
database	PostgreSQL 17 Alpine	5432	Healthy

7. Integración y despliegue continuo

El workflow `.github/workflows/ci.yml` se ejecuta en push y pull request hacia main, además de admitir ejecución manual. Los trabajos de frontend y backend se ejecutan en paralelo; la construcción Docker espera que ambos terminen correctamente y la publicación solo se activa en un push a main.



Trabajo	Acciones principales	Condición
frontend	checkout, setup-node, npm install, lint, build	Push y PR
backend	checkout, setup-node, npm ci, npm test	Push y PR
docker	Buildx y construcción de dos imágenes	Después de validar
publish	Login y push de etiquetas latest/SHA	Push a main

Secretos requeridos en GitHub

- DOCKER_USERNAME: nombre de usuario de Docker Hub.
- DOCKER_PASSWORD: access token de Docker Hub.

La ejecución remota y la publicación quedan preparadas, pero requieren subir los cambios al repositorio y configurar estos secretos. No se incluyeron credenciales en el código ni en archivos versionables.

8. Pruebas y evidencias

Prueba	Resultado observado	Estado
ESLint	Sin errores	APROBADA
Build PWA	638 módulos; sw.js y precaché generados	APROBADA
Test backend	1 prueba ejecutada, 1 aprobada	APROBADA
Docker build	Dos imágenes construidas	APROBADA
Docker Compose	Tres servicios en estado healthy	APROBADA
Registro UI/API	Usuario creado correctamente	APROBADA
Login JWT	Token recibido y ruta protegida habilitada	APROBADA
CRUD API	GET, POST, PUT y DELETE exitosos	APROBADA
CRUD navegador	Crear, editar, confirmar y eliminar	APROBADA
Responsive	Viewport 390 x 844 sin overflow horizontal	APROBADA

9. Resultados obtenidos

- Aplicación React instalable como PWA y ejecutable en modo standalone.
- Registro e inicio de sesión conectados con PostgreSQL y protegidos por JWT.
- Módulo de productos completamente funcional y accesible solo con sesión.
- Service Worker, manifiesto e iconos generados y servidos correctamente.
- Frontend, backend y base de datos ejecutándose en contenedores saludables.
- Pipeline de GitHub Actions listo para validar y publicar imágenes.

10. Conclusiones

La práctica permitió integrar desarrollo frontend, seguridad de API, persistencia y automatización en una única solución reproducible. La centralización de Axios y el middleware JWT reducen duplicación y aseguran que los recursos de productos no sean accesibles sin autenticación.

La contenerización elimina diferencias entre entornos y el uso de health checks garantiza que cada servicio inicie únicamente cuando sus dependencias están disponibles. El workflow separa validación, construcción y publicación, lo que facilita identificar errores en los registros de GitHub Actions.

Las pruebas funcionales ejecutadas tanto por API como por navegador demostraron el flujo completo: registro, login, almacenamiento del JWT, acceso protegido y operaciones CRUD. La PWA conservó un diseño responsive en escritorio y móvil.

11. Recomendaciones

- Cambiar JWT_SECRET y las credenciales de PostgreSQL antes de producción.
- Usar un access token de Docker Hub en lugar de la contraseña de la cuenta.
- Revisar los logs de Actions después de cada push a main.
- Aplicar HTTPS en un despliegue público para mantener las garantías de PWA.
- Rotar secretos y nunca confirmar archivos .env en Git.
- Mantener React Router actualizado. npm reporta dos avisos asociados al modo RSC; este proyecto es una SPA y no utiliza RSC, pero el seguimiento sigue siendo recomendable.

Estado final

Implementación local	Completa y validada
Contenedores	Construidos y saludables
Workflow CI/CD	Configurado
Publicación externa	Pendiente de push y secretos del propietario